Actividad I. Generación, ordenamiento y medición de tiempo en ATmega328P

Flores Mondragón José Adrián, Tapia Ortega Citlalli Yael

10 de marzo de 2026.

Resumen

En este informe se explica la función de cada bloque del programa, se resume la teoría, se desarrolla el pseudocódigo y se justifican las decisiones de diseño tomadas en el código. En el proyecto se usa el lenguaje ensamblador para el microcontrolador ATmega328P para una secuencia que genera 100 números pseudoaleatorios en el intervalo de 0 a 100, almacenamiento de los valores en memoria SRAM, y su ordenamiento ascendente mediante dos algoritmos clásicos: Bubble Sort e Insertion Sort. Además, se agrega una medición con Timer1 para estimar el tiempo de ejecución de cada algoritmo. El programa está estructurado en subrutinas pequeñas, para facilitar la lectura. La separación entre la tabla original y las tablas ordenadas permite hacer la comparación.

1. Introducción

Los algoritmos de ordenamiento son una herramienta importante ya que son la base de operaciones como búsqueda, clasificación y organización de estructuras de datos. La elección de Bubble Sort e Insertion Sort responde a que ambos son algoritmos sencillos de explicar, pero muestran complejidad teórica.

El uso de Timer1 brinda una medición simple sin necesidad de hardware adicional. Actualmente existen algoritmos más eficientes para grandes volúmenes de información, estos métodos clásicos continúan siendo valiosos en sistemas donde el objetivo es comprender el costo de cada operación y su implementación a bajo nivel.

2. Marco teórico

2.1. Algoritmos de ordenamiento

Ordenar datos significa reorganizar un conjunto de elementos de acuerdo con un criterio definido. Esta tarea aparece en aplicaciones diversas como la búsqueda eficiente de registros, la organización de mediciones de sensores, el procesamiento de eventos, el manejo de colas de prioridad y la preparación de datos para transmisión o almacenamiento.

En este proyecto el objetivo es observar de manera tangible cómo una estrategia algorítmica se convierte en instrucciones reales y cómo esa implementación afecta el tiempo de ejecución.

2.2. Notación Big-O

La notación expresa el crecimiento del costo de un algoritmo cuando aumenta el tamaño de la entrada. No mide directamente segundos o microsegundos, sino la tendencia general del número de operaciones que el algoritmo necesita conforme el problema se hace más grande. Esto permite comparar métodos entre sí sin depender por completo de una computadora o microcontrolador específico.

Si un algoritmo $O(n)$ incrementa su costo de forma proporcional a la entrada. Un algoritmo $O(n^2)$, crece mucho más rápido, ya que el número de operaciones se relaciona con el cuadrado del número de datos. Insertion Sort suele aprovechar mejores arreglos parcialmente ordenados y por ello puede superar a Bubble Sort en muchos escenarios reales.

Algorit	Mejor caso	Caso promedio	Peor caso	Observación
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	En esta implementación no se incluye bandera de intercambio; por ello su comportamiento sigue siendo cuadrático.
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Puede ser muy eficiente si los datos ya están casi ordenados, porque desplaza solo los elementos necesarios.
Gen. pseudoaleatoria	$O(n)$	$O(n)$	$O(n)$	Se genera un número por iteración y se almacena una sola vez.

2.3. Generación pseudoaleatoria con LFSR de 8 bits

El código utiliza un registro de desplazamiento con realimentación lineal como generador pseudoaleatorio de bajo costo. El estado interno se almacena en un byte llamado rng_state. En cada llamada se toma el bit menos significativo, se desplaza a la derecha y, si el bit era uno, se aplica una operación XOR con la constante 0xB8. Esto genera una secuencia útil para pruebas de laboratorio, simulaciones y comparación de algoritmos.

Al actualizar el estado, el valor se reduce al intervalo de 0 a 100 mediante restas sucesivas de 101. Esto evita el uso de divisiones, que serían más costosas. Como el estado del LFSR es de 8 bits, la reducción por restas mantiene el código sencillo de comprender.

2.4. Medición temporal con Timer1

Se consideró la frecuencia de reloj de 16 MHz, cada incremento del contador TCNT1 ocurre cada 4 microsegundos (64 / 16 000 000 = 4×10^{-6} s). Por tanto, el tiempo aproximado de cada algoritmo puede estimarse mediante la relación: tiempo ≈ ticks × 4 µs.

La medición no es un sistema de alta precisión. Su finalidad es mostrar una relación entre el número de operaciones ejecutadas y el valor leído del temporizador. La estrategia es útil porque permite comparar dos algoritmos bajo las mismas condiciones, aun con el costo del llamado a las subrutinas.

3. Descripción del proyecto

El código se apoya en el archivo que proporciona nombres simbólicos para registros, bits y direcciones del dispositivo.

3.1. Organización de memoria

SRAM	**Tamaño**	**Función**	
table_of_unsorted_numbers	100 bytes	Almacena la secuencia original de prueba	Debe permanecer sin alteración para que ambos algoritmos trabajen sobre la misma entrada.
table_of_sorted_numbers_alg1	100 bytes	Copia destinada a Bubble Sort	Permite verificar de manera independiente el resultado del algoritmo 1.
table_of_sorted_numbers_alg2	100 bytes	Copia destinada a Insertion Sort	Permite verificar de manera independiente el resultado del algoritmo 2.
rng_state	1 byte	Estado interno del LFSR	La semilla inicial determina la secuencia

			pseudoaleatoria.
time_alg1_ticks	2 bytes	Valor medido de TCNT1 para Bubble Sort	Se guarda en formato LSB/MSB.
time_alg2_ticks	2 bytes	Valor medido de TCNT1 para Insertion Sort	Se guarda en formato LSB/MSB.

3.2. Flujo del programa

i. Inicia Stack Pointer y limpia r1.
ii. Configura el Timer1 en modo normal.
iii. Carga de la semilla inicial del generador pseudoaleatorio.
iv. Llena tabla original con 100 valores entre 0 y 100.
v. Copia la tabla original a la memoria del algoritmo 1.
vi. Reinicio Timer1, ejecución de Bubble Sort y almacena el tiempo medido.
vii. Copia la tabla original a la memoria del algoritmo 2.
viii. Reinicio Timer1, ejecución de Insertion Sort y almacena el tiempo medido.
ix. Entrada lazo infinito = estado final estable y fácil de inspeccionar.

3.3. Diseño

Se escogió un LFSR en lugar de una rutina pseudoaleatoria más compleja porque su implementación en ensamblador es compacta, rápida y suficiente para generar datos de prueba. Se usaron dos arreglos auxiliares de salida para asegurar una comparación justa entre algoritmos, ya que, si se aplicara el segundo ordenamiento sobre la salida del primero, el experimento quedaría sesgado.

Bubble Sort e Insertion Sort se eligieron porque son algoritmos clásicos, de estructura clara y de fácil rastreo. Aunque existen algoritmos más eficientes, como Merge Sort o Quick Sort, su implementación en ensamblador añade complejidad estructural, especialmente en el manejo de particiones o memoria auxiliar.

La medición de tiempo se realiza con Timer1 porque ofrece una referencia hardware coherente y simple. De igual manera, se dejó el programa en un lazo infinito final para facilitar la revisión de la SRAM sin que el flujo vuelva a sobrescribir datos. Esto es conveniente, ya que permite detener la simulación en un punto estable.

4. Código

<u>Bloques</u>

RESET: Prepara al sistema. Carga la dirección final de la SRAM en el Stack Pointer para que las instrucciones RCALL y RET funcionen correctamente. Limpia el registro r1, configura Timer1 en modo normal y establece la frecuencia del reloj del microcontrolador de 64. Finalmente inicializa el generador pseudoaleatorio y coordina las llamadas a las subrutinas de llenado, copia, ordenamiento y lectura de tiempo.

DONE: El lazo infinito final mantiene el programa detenido en un estado estable facilitando revisar el contenido de la memoria, las tablas ya ordenadas y los tiempos capturados sin que se reinicie el proceso.

<u>Subrutinas</u>

TIMER1_RESET: Su trabajo es sencillo pero importante puesto que coloca ceros en los registros TCNT1H y TCNT1L antes de comenzar cada medición. Si no se hace los tiempos guardados

serían acumulativos y la comparación quedaría invalidada.

TIMER1_READ_TO_time_alg1 y TIMER1_READ_TO_time_alg2: Ambas leen el valor de TCNT1 al final de cada ordenamiento y lo almacenan en dos bytes consecutivos de la SRAM. Tienen dos nombres distintos para que la lectura del código sea clara y para que cada resultado tenga una ubicación explícita en memoria.

COPY_N_BYTES: Esta se realiza con punteros y un contador decreciente. Lee un byte desde la dirección apuntada por Z y escribe el mismo valor en la dirección que señala Y. Esto evita repetir lógica y asegura que la tabla original pueda usarse para ambos algoritmos.

FILL_RANDOM_0_100: Esta dirige el puntero al arreglo original, fija un contador de 100 elementos y llama repetidamente al generador pseudoaleatorio. El resultado de cada llamada se deja en r16 y se almacena en la SRAM. Con esto la tabla de prueba queda llena al inicio del experimento.

RNG_NEXT_0_100: Lee el estado actual, aísla el bit menos significativo, desplaza el byte una posición a la derecha y, si el bit aislado era uno, aplica una XOR con 0xB8. El nuevo estado se guarda de nuevo en rng_state. Reduce el valor al intervalo 0..100 restando 101 tantas veces como sea necesario. El resultado final regresa a r16.

BUBBLE_SORT_N: Bubble Sort realiza recorridos sucesivos sobre el arreglo. En cada posición compara dos elementos adyacentes: si el primero es mayor que el segundo, los intercambia. Se reutiliza el registro Y como puntero móvil para facilitar el acceso a arr[j] y arr[j+1] mediante instrucciones ld, ldd, st y std.

INSERTION_SORT_N: Insertion Sort considera que la primera parte del arreglo ya está ordenada. Toma el valor llamado key y lo inserta en la posición correcta, desplazando a la derecha todos los elementos mayores. Requiere recalcular direcciones para i y j, el algoritmo requiere menos movimientos que Bubble Sort cuando la secuencia está cerca de estar ordenada.

5. Instrucciones AVR

Instrucción / recurso	*Por qué se usó*	**Alternativa**	*Por qué no se eligió*
LDS / STS	Permiten acceder a registros y variables ubicados en espacio de datos o I/O extendido.	**IN / OUT**	Solo funcionan con direcciones de I/O directo. TCCR1A, TCCR1B o TCNT1 son de espacio extendido y no son adecuados.
LD / ST con punteros	Facilitan recorrer arreglos en SRAM de forma compacta.	**Accesos absolutos con LDS/STS repetidos**	Harían el código más largo y menos flexible para recorrer tablas.
LDD / STD con Y+desplazamiento	Permiten leer y escribir arr[j+1] sin recalcular toda la dirección.	**Incrementar/decrementar punteros constantemente**	Complica el seguimiento y aumenta instrucciones auxiliares.
MOVW	Copia pares de registros de 16 bits de una sola vez, ideal para punteros.	**MOV individual de bytes alto y bajo**	Más instrucciones y menor claridad al copiar direcciones base.
ADIW	Incrementa punteros de 16 bits con sintaxis simple.	**INC byte bajo y manejo manual de acarreo**	Sería más propenso a errores y menos legible.
CP + BRSH / BRLO	Implementan comparaciones	**Comparar con lógica manual de resta y banderas**	Sería menos directo.

	ordenadas entre valores sin signo.		
RCALL / RET	Modularizan el código y facilitan la reutilización.	**Expandir todo inline**	Sería más largo, más difícil de depurar y menos didáctico.
Reducción por restas de 101	Es simple y suficiente para un rango pequeño.	**División o modulo**	La división complica la rutina y no da ventaja para un valor de 8 bits.

6. Pseudocódigo de las rutinas principales

Generación pseudoaleatoria:

```
Para i = 0 hasta 99
    state ← actualizar_LFSR(state)
    mientras state ≥ 101 hacer
        state ← state − 101
    fin mientras
    tabla_original[i] ← state
Fin para
```

Bubble Sort:

```
    Para i = N−1 hasta 1
        Para j = 0 hasta i−1
            si A[j] > A[j+1] entonces
                intercambiar A[j],
    A[j+1]
            fin si
        Fin para
    Fin para
```

Insertion Sort:

```
Para i = 1 hasta N−1
    key ← A[i]
    j ← i−1
    mientras j ≥ 0 y A[j] > key hacer
        A[j+1] ← A[j]
        j ← j−1
    fin mientras
    A[j+1] ← key
Fin para
```

7. Análisis de resultados

SRAM	
0x0100	table_of_unsorted_numbers
0x0164	table_of_sorted_numbers_alg1
0x01C8	table_of_sorted_numbers_alg2
0X022C	time_alg1_ticks
0X022E	time_alg2_ticks
0X230	rng_state

Las estructuras de datos del programa se almacenan en la memoria SRAM del microcontrolador ATmega328P. La tabla original y las dos tablas de ordenamiento ocupan 100 bytes cada una, lo cual corresponde al número de elementos generados. Estas tablas se almacenan de forma consecutiva en memoria facilitan el acceso mediante los registros puntero (Z y Y).

En esta práctica se implementaron los algoritmos Bubble Sort e Insertion Sort en el microcontrolador ATmega328P para ordenar 100 números generados de forma pseudoaleatoria. Al revisar las tablas en memoria se comprobó que ambos algoritmos ordenan correctamente los datos. También se utilizó el Timer1 para observar el tiempo aproximado que tarda cada método. En general, los resultados coinciden con lo esperado teóricamente, ya que ambos algoritmos tienen una complejidad $O(n^2)$, aunque en algunos casos Insertion Sort puede ser un poco más rápido. La práctica permitió entender mejor cómo funcionan estos algoritmos cuando se implementan directamente en un microcontrolador.

Referencias

[1] Microchip Technology Inc., ATmega328/P Data Sheet, Microchip, 2018.

[2] Khan Academy, “Notación Big-O y eficiencia de algoritmos,” *Khan Academy*. [En línea]. Disponible en: https://es.khanacademy.org/computing/computer-science/algorithms

[3] GeeksforGeeks, “Analysis of Algorithms | Big-O Analysis,” *GeeksforGeeks*, 2023. [En línea]. Disponible en: https://www.geeksforgeeks.org/analysis-of-algorithms-big-o-analysis/

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.